

FIELD PROJECT SYNOPSIS

Project Title:- Serverless Application Development [Cloud-Based Student Complaint & Request Management System Using Serverless Architecture.

Name: Shravan Sasane

Class: TY cloud computing

Roll no: 47

GUIDE by: 1) Priyamvada Patil Mam.

2) Mayure Taware sir.

1. Introduction:

Educational institutions handle various student complaints and requests related to academics, administration, facilities, technical issues, and other student services. In a conventional process, such complaints may be submitted through physical forms, emails, or informal communication channels, which can make tracking, status management, and response handling difficult.

This project proposes a cloud-based Student Complaint & Request Management System for **New Arts, Commerce and Science College**. The system will provide a centralized web-based platform through which students can submit complaints or requests and track their status, while authorized administrators can review, manage, update, and respond to them.

The application will be developed using a **serverless architecture on Amazon Web Services (AWS)**. The design follows the Field Project requirement of developing cloud solutions that demonstrate architecture design, application deployment, security, monitoring, cost management, testing, and disaster recovery considerations.

2. Problem Statement:

Students need a simple and centralized mechanism to submit complaints and requests and determine their current status. A manually managed process can make it difficult to maintain complaint records, track pending requests, provide timely responses, and maintain an organized history.

Therefore, there is a need for a secure and scalable cloud-based system that can manage student complaints and requests through a centralized platform.

3. Objectives:

The main objectives of the project are:

1. To develop a centralized web-based platform for submitting student complaints and requests.
 2. To provide secure authentication for students and administrators.
 3. To allow students to view their submitted complaints and track their status.
 4. To allow administrators to view, filter, update, and respond to complaints and requests.
 5. To implement a serverless cloud architecture using AWS services.
 6. To store application data securely using a managed cloud database.
 7. To implement role-based access control and least-privilege permissions.
 8. To monitor application activity, errors, and performance using cloud monitoring services.
 9. To implement Infrastructure as Code for reproducible cloud infrastructure deployment.
 10. To perform functional, security, performance, and disaster recovery testing.
 11. To analyze and optimize the cloud resources and operating cost.
-

4. Proposed System:

The proposed system will contain two primary user roles:

Student:-

Students will be able to:

- Register and log in securely.
- Submit a complaint or request.
- Select an appropriate category.
- Provide a subject and description.
- View their submitted complaints and requests.
- Track the current status.

- View responses provided by the administrator.

Administrator:-

Authorized administrators will be able to:

- Log in securely.
- View submitted complaints and requests.
- Filter records according to status or category.
- View complete complaint details.
- Change complaint status.
- Add an administrative response or resolution.
- View basic complaint statistics.

The application will use a serverless backend so that compute resources are managed by the cloud provider instead of requiring dedicated application servers.

5. System Architecture:

The proposed system follows a **serverless cloud architecture on Amazon Web Services (AWS)**. The system consists of a web frontend, authentication layer, API layer, serverless backend, database, security, and monitoring components:

The main architecture flow is:

User → Amazon S3 → Amazon Cognito → API Gateway → AWS Lambda → DynamoDB → Response

AWS Service	Purpose
Amazon S3	Hosting the static web frontend
Amazon Cognito	Student and administrator authentication
Amazon API Gateway	Provides API endpoints for the application
AWS Lambda	Executes backend application logic
Amazon DynamoDB	Stores student complaint and request data
AWS IAM	Controls permissions and access to AWS resources
Amazon CloudWatch	Monitoring, metrics, and application logs

AWS Service	Purpose
Terraform	Infrastructure as Code for cloud resources

The serverless architecture is consistent with the course program's Topic 4 description, which identifies Lambda, API Gateway, and DynamoDB as the core components.

6. Basic System Workflow:

The general workflow of the proposed system will be:

Student → Web Application → Authentication → API Gateway → Lambda → DynamoDB

For administrators:

Administrator → Web Application → Authentication → API Gateway → Lambda → DynamoDB

When a student submits a complaint, the request will be sent through the API Gateway to an AWS Lambda function. Lambda will validate and process the request and store the complaint information in DynamoDB. Administrators will retrieve and manage the records through appropriate API operations.

7. Advantages of the Project:

1. **Easy complaint submission:** Students can submit complaints or requests online without visiting the administration office.
 2. **Faster tracking:** Students can check the current status of their complaint, such as Pending, In Progress, or Resolved.
 3. **Centralized management:** Admin can view, manage, update, and respond to all complaints from one system.
 4. **Serverless and scalable:** AWS manages the underlying servers, and the system can handle changing workloads without manually managing servers.
 5. **Secure authentication:** Amazon Cognito provides user login and helps separate Student and Admin access.
 6. **Low infrastructure management:** Services like Lambda, DynamoDB, and API Gateway reduce the need to maintain traditional servers and databases.
 7. **Monitoring and logging:** CloudWatch can help monitor the application and identify errors.
 8. **Infrastructure as Code:** Terraform makes AWS infrastructure easier to create, reproduce, and manage consistently.
-

8. Security:

Security will be implemented at multiple levels.

- Authentication will be handled using Amazon Cognito.
- Student and administrator access will be separated according to their roles.
- AWS IAM roles and policies will follow the principle of least privilege.
- Application communication will use secure HTTPS endpoints.
- Lambda functions will receive only the permissions required for their operations.
- Input validation will be implemented to reduce invalid or unsafe requests.
- CloudWatch logs will be used to monitor application activity and errors.

The Field Project handbook identifies IAM, encryption, least privilege, and monitoring as important cloud architecture and security considerations.

9. Monitoring and Logging:

Amazon CloudWatch will be used to monitor the serverless application.

The project will monitor:

- Lambda invocations
- Lambda errors
- Lambda execution duration
- API Gateway requests
- API errors
- Application logs
- Relevant performance metrics

This will help identify failures and performance issues during testing and operation.

10. Testing:

The proposed system will be tested using different categories:

Functional Testing:

Testing registration, login, complaint submission, complaint history, status updates, administrator functions, and responses.

Security Testing:

Testing role-based access and verifying that students cannot access administrator functions or other students' complaint records.

Performance Testing:

Measuring API response behaviour, Lambda execution duration, and error rates under controlled requests.

Disaster Recovery Testing:

Testing the recovery of important application data and validating that the system can be restored after a controlled failure scenario.

The handbook specifically includes functional, performance, security, load, and disaster recovery testing as part of the project lifecycle.

11. Backup and Disaster Recovery:

The project will include a disaster recovery approach for the application's stored data and cloud infrastructure.

DynamoDB backup and restore capabilities will be considered for data recovery. Terraform configuration and application source code will also provide a reproducible method for rebuilding the cloud infrastructure.

Recovery testing will be documented using suitable Recovery Time Objective (RTO) and Recovery Point Objective (RPO) targets defined during implementation.

12. Infrastructure as Code[optional]:

Terraform will be used to define and provision the required cloud infrastructure.

Infrastructure configuration will be maintained as code so that the project resources can be recreated in a controlled and consistent manner. This will also provide evidence for the Infrastructure as Code requirement of the Field Project.

13. Cost Analysis:

The project will focus on a low-cost serverless architecture using managed AWS services.

The following will be analyzed:

- AWS services used

- Resource usage
- Estimated cloud cost
- Actual usage where applicable
- Free Tier eligibility where applicable
- Cost optimization opportunities

The final cost analysis will be based on the actual architecture and usage rather than assumed production-scale figures.

14. Expected Outcomes:

The expected outcome is a functional cloud-based student complaint and request management system that:

- Provides a centralized platform for complaints and requests.
 - Allows students to track complaint status.
 - Allows administrators to manage and respond to complaints.
 - Uses a serverless AWS architecture.
 - Provides secure authentication and role-based access.
 - Stores application data using a managed database.
 - Provides monitoring and logging.
 - Supports infrastructure deployment through Terraform.
 - Demonstrates testing, backup/recovery, and cost analysis.
-

15. Scope:

The project will focus on complaint and request management for students and authorized administrators.

The initial scope includes authentication, complaint/request submission, complaint history, status tracking, administrator management, responses, basic statistics, security, monitoring, testing, Infrastructure as Code, backup/recovery planning, and cost analysis.

Advanced features such as mobile applications, SMS notifications, complex analytics, and large-scale multi-region deployment are outside the initial scope and may be considered as future enhancements.

16. Conclusion:

The proposed **Cloud-Based Student Complaint & Request Management System Using Serverless Architecture** will demonstrate how AWS serverless services can be used to build a secure, scalable, and manageable cloud application for an educational environment. The project will cover not only application development but also cloud architecture, security, monitoring, Infrastructure as Code, testing, disaster recovery, and cost analysis in accordance with the Field Project requirements.